Template Attacks

Wei-Chih Hong

Outline

- General description of SPA
- Visual inspections of power traces
- Template attack
- Template-based DPA

Simple Power Analysis

- “SPA is a technique that involves directly interpreting power consumption measurements collected during cryptographic operations,” by Kocher.

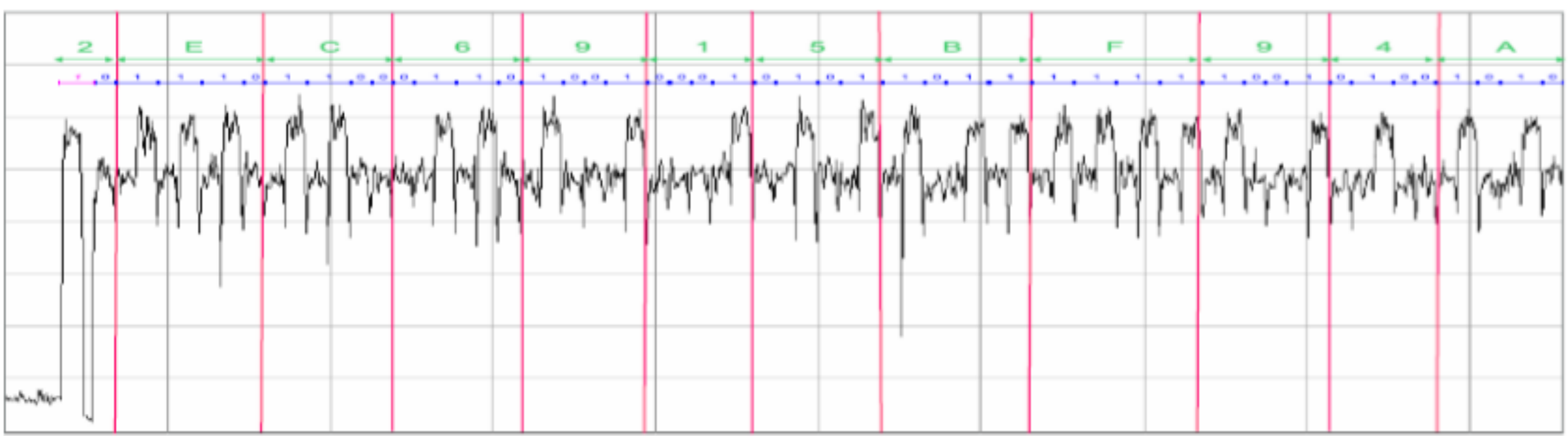
- Only one or a few traces are available
- Exploiting key-dependent patterns within a trace
- Requiring detailed knowledge about the implementation

Visual Inspection of Power Traces

- **Possibility of distinguishing instructions within a trace**
  - E.g. different instructions require different numbers of cycles, different amounts of energies
- **This fact may lead to security problems if the sequence of instructions directly depends on the key.**

SPA Example

- RSA exponentiation $M = C^d \bmod N$
- Square-and-multiply algorithm



Example for Software Implementation of AES

- `SETB`, `MOV`, `XRL` require 12 clock cycles whereas the other instructions require 24.

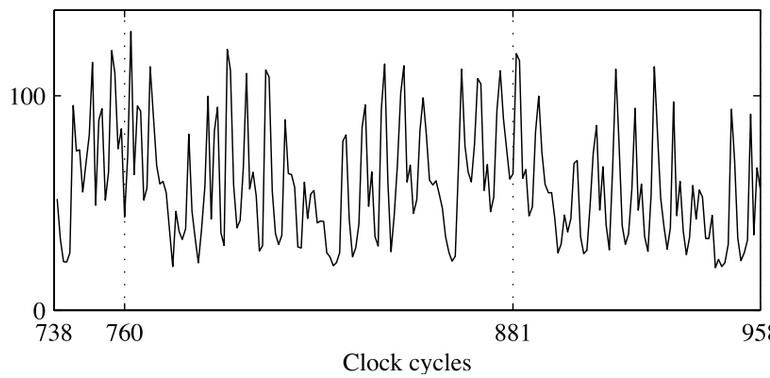


Figure 5.3. The sequence of AddRoundKey, SubBytes, and ShiftRows operations.

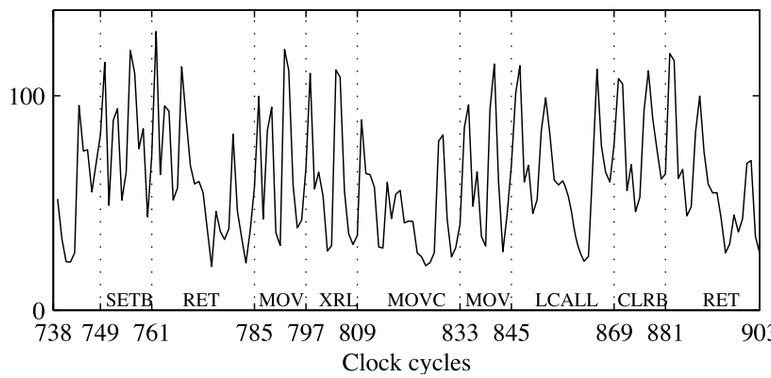


Figure 5.4. The annotated sequence of AddRoundKey, SubBytes, and ShiftRows operations.

```
LCALL SET_ROUND_TRIGGER
MOV A,ASM_input + 0     ; load a0
XRL A,ASM_key + 0       ; add k0
MOVC A,@A + DPTR        ; S-box look-up
MOV ASM_input, A        ; store a0
LCALL CLEAR_ROUND_TRIGGER
```

Figure 5.5. The sequence of assembly instructions that corresponds to Figure 5.4.

Template Attacks

- **Assumptions**
  - The attacker gets hold of the cryptographic device and has full control over the device.
  - The power traces can be modeled by a multivariate normal distribution.
- **Two-phase attack**
  - Template building
  - Template matching

What is a “Template”?

- **Multivariate Normal (Gaussian)**

$$f(\mathbf{x}) = \frac{1}{\sqrt{(2\pi)^n det(\mathbf{C})}} exp\left(-\frac{1}{2}(\mathbf{x}-\mathbf{m})' \cdot \mathbf{C}^{-1} \cdot (\mathbf{x}-\mathbf{m})\right)$$

$$\mathbf{x} = (x_1, x_2, \ldots)'$$

$$\mathbf{m} = (m_1, m_2, \ldots)'$$ mean vector

$$\mathbf{C} = \begin{pmatrix} c_{11} & c_{12} & \cdots \\ c_{21} & c_{22} & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix}$$ covariance matrix

- **A template is a pair of $(\mathbf{m}, \mathbf{C})$ under certain hypothesis (condition).**

Template Building

- **Collecting and grouping traces of different targets of the templates**
  - Pairs of data and key $\hbar_{d_i,k_j} = (\mathbf{m}, \mathbf{C})_{d_i,k_j}$
  - Intermediate values $\hbar_{v_{ij}}$, with $v_{ij} = S(d_i \oplus k_j)$
  - Templates with power models $\hbar_{HW(v_{ij})}$
- **Estimating mean vectors and covariance matrices of the interesting points**

Template Matching

- Given a power trace $\mathbf{t}$, we compute the following probability for each template h_i.

$$p(\mathbf{t}; (\mathbf{m}, \mathbf{C})_{h_i}) = \frac{exp\left(-\frac{1}{2}(\mathbf{t}-\mathbf{m})' \cdot \mathbf{C}^{-1} \cdot (\mathbf{t}-\mathbf{m})\right)}{\sqrt{(2\pi)^n det(\mathbf{C})}}$$

- Decide h_l if we have

$$p(\mathbf{t}; (\mathbf{m}, \mathbf{C})_{h_l}) > p(\mathbf{t}; (\mathbf{m}, \mathbf{C})_{h_i}) \forall i \neq l$$

  - Maximum likelihood

Template Matching

- To avoid the exponentiation

$$\ln p(\mathbf{t};(\mathbf{m},\mathbf{C})) = -\frac{1}{2}(\ln((2\cdot\pi)^{N_{IP}}\cdot det(\mathbf{C}))+ (\mathbf{t}-\mathbf{m})'\cdot\mathbf{C}^{-1}\cdot(\mathbf{t}-\mathbf{m}))$$

- Find h_l such that

$$|\ln p(\mathbf{t};(\mathbf{m},\mathbf{C})_{h_l})| < |\ln p(\mathbf{t};(\mathbf{m},\mathbf{C})_{h_i})| \quad \forall i \neq l$$

Reduced Template

- Consists of the mean vector $\mathbf{m}$ only

$$p(\mathbf{t};\mathbf{m}) = \frac{exp\left(-\frac{1}{2}(\mathbf{t}-\mathbf{m})' \cdot (\mathbf{t}-\mathbf{m})\right)}{\sqrt{(2\pi)^{N_{IP}}}}$$

$$\ln p(\mathbf{t};\mathbf{m}) = -\frac{1}{2}(\ln(2\cdot\pi)^{N_{IP}} + (\mathbf{t}-\mathbf{m})' \cdot (\mathbf{t}-\mathbf{m}))$$

  - Find h_l such that

$$(\mathbf{t}-\mathbf{m}_{h_l})' \cdot (\mathbf{t}-\mathbf{m}_{h_l}) < (\mathbf{t}-\mathbf{m}_{h_i})' \cdot (\mathbf{t}-\mathbf{m}_{h_i}) \quad \forall i \neq l$$

  - Least-square test

Example for a MOV Instruction

- Moving a byte with different Hamming weights

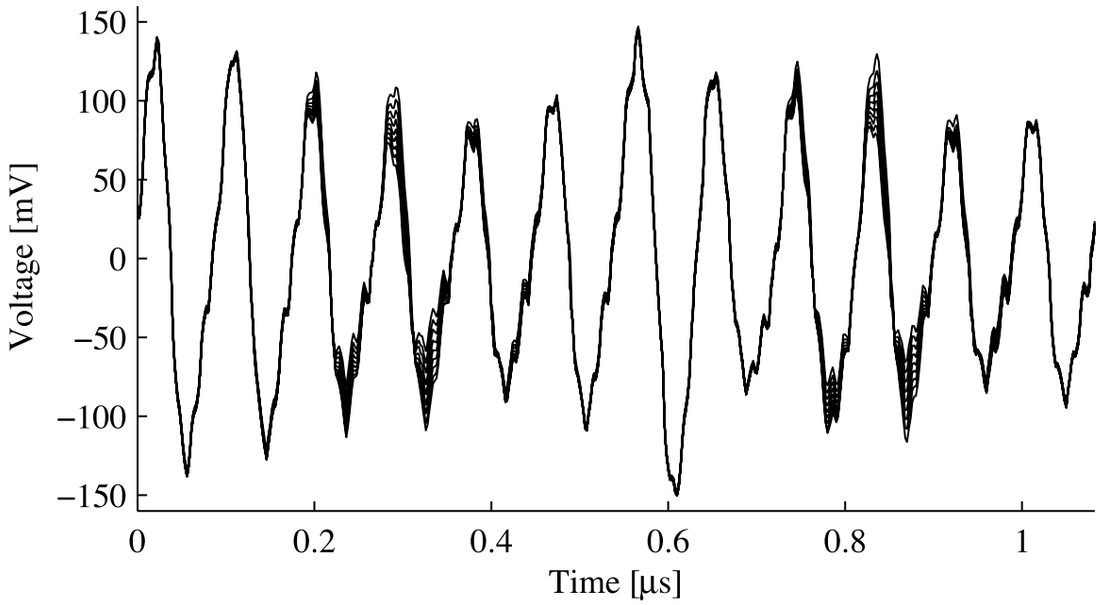


Figure 5.6. Power traces of MOV instructions with different Hamming weights.

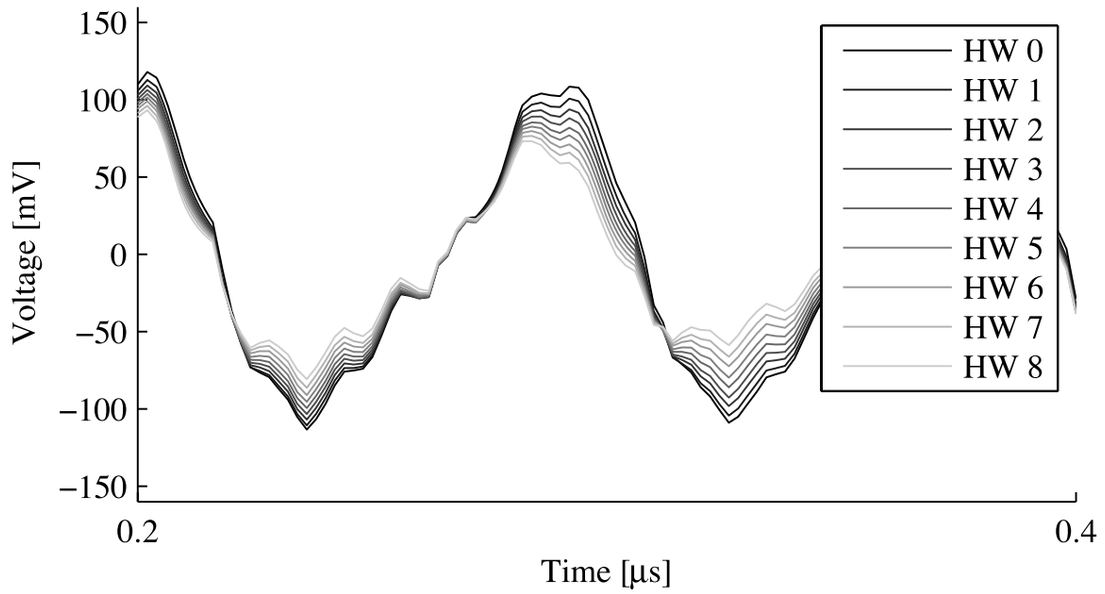


Figure 5.7. Zoomed in on the MOV instructions with different Hamming weights.

Example for a MOV Instruction

- Template h_0 with 5 interesting points

$$\mathbf{C_0} = \begin{pmatrix} 1.77 & 1.03 & 0.29 & 0.22 & 0.34 \\ 1.03 & 2.71 & 0.50 & 0.54 & 0.75 \\ 0.29 & 0.50 & 1.88 & 0.17 & 0.22 \\ 0.22 & 0.54 & 0.17 & 1.32 & 0.21 \\ 0.34 & 0.75 & 0.22 & 0.21 & 1.56 \end{pmatrix}$$

$$\mathbf{m_0} = (-81.52, -78.65, 70.93, 74.01, 49.21)'$$

- 9 traces, corresponding to 9 Hamming weights

$$\mathbf{T} = \begin{pmatrix} -81.20 & -77.70 & 71.20 & 74.30 & 49.70 \\ -78.80 & -74.10 & 68.60 & 66.60 & 45.10 \\ -76.30 & -71.20 & 67.10 & 63.40 & 44.30 \\ -74.20 & -68.50 & 65.60 & 59.30 & 42.80 \\ -72.40 & -64.20 & 63.50 & 56.00 & 41.30 \\ -71.90 & -62.70 & 59.00 & 50.10 & 37.70 \\ -70.80 & -60.00 & 57.70 & 44.90 & 36.00 \\ -65.50 & -53.50 & 58.20 & 44.60 & 37.80 \\ -60.60 & -45.90 & 56.70 & 40.20 & 37.00 \end{pmatrix}$$

Example for a MOV Instruction

- Template matching

$$
\begin{aligned}
\ln p(\mathbf{t}'_1; \hbar_0) &= -5.98 \\
\ln p(\mathbf{t}'_2; \hbar_0) &= -49.54 \\
\ln p(\mathbf{t}'_3; \hbar_0) &= -99.36 \\
\ln p(\mathbf{t}'_4; \hbar_0) &= -182.09 \\
\ln p(\mathbf{t}'_5; \hbar_0) &= -295.03 \\
\ln p(\mathbf{t}'_6; \hbar_0) &= -486.59 \\
\ln p(\mathbf{t}'_7; \hbar_0) &= -681.45 \\
\ln p(\mathbf{t}'_8; \hbar_0) &= -796.20 \\
\ln p(\mathbf{t}'_9; \hbar_0) &= -1\,131.03
\end{aligned}
$$

- The trace with zero Hamming weight has the highest value, i.e. highest probability.

Template Attack on Key Schedule

```
RC[1..10] = ('01','02','04','08','10','20','40','80','1B','36')
Rcon[i]   = (RC[i],'00','00','00')

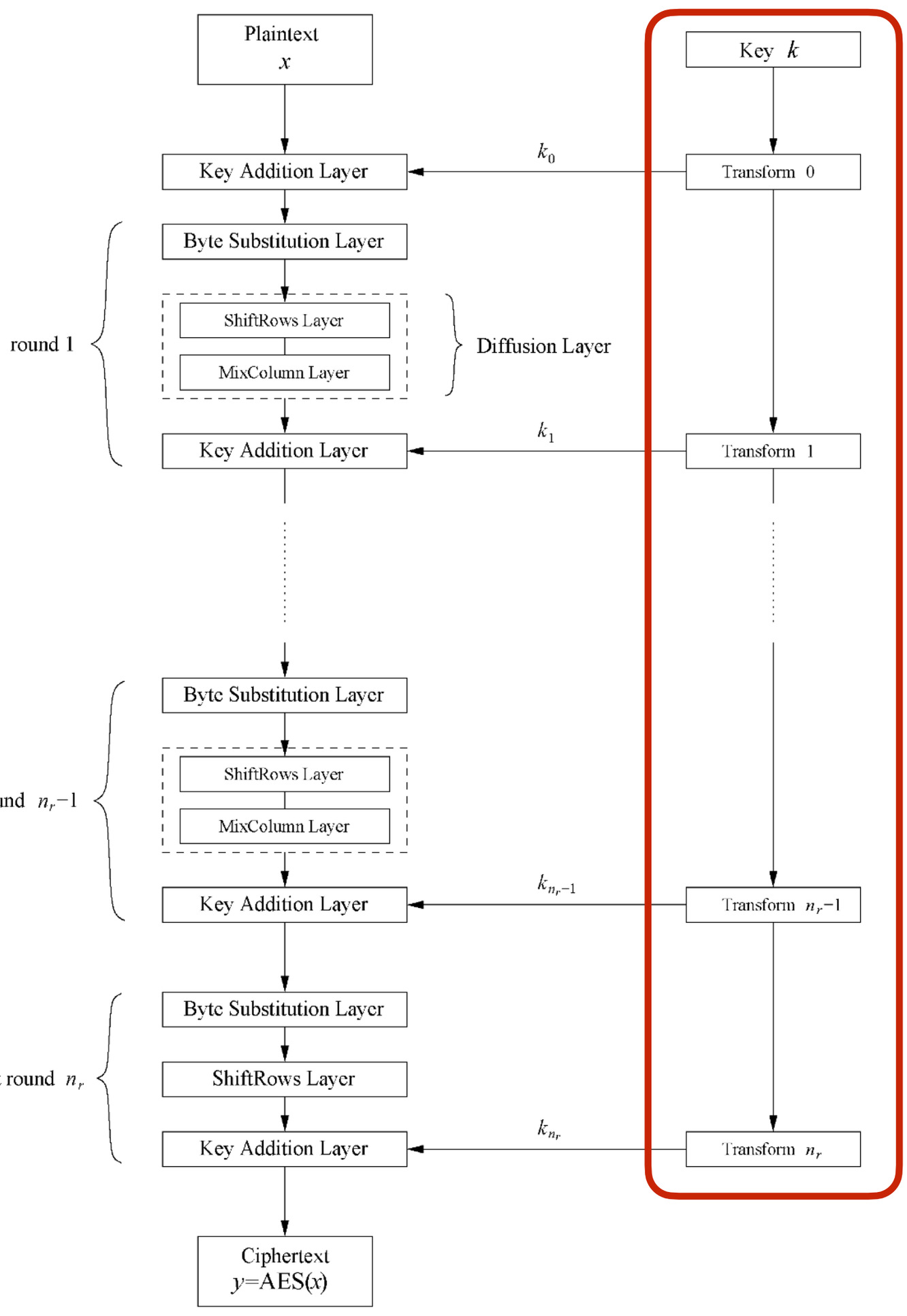
for(i = 0; i < 4; i++)
{
   W[i] = (key[4*i], key[4*i+1], key[4*i+2], key[4*i+3])
}
for(i = 4; i < 44; i++)
{
   temp = W[i-1]
   if (i mod 4 == 0)
       temp = SubWord(RotWord(temp)) xor Rcon[i/4]
   W[i] = W[i-4] xor temp
}
```

Figure B.8. Pseudo code for the AES key expansion.

If about 81 Hamming weights are used in such an attack, the average number of keys that remain for a brute-force search is 11.

Table 5.1. Observing the Hamming weights of intermediate values allows reducing the brute-force search space drastically. The only pairs ($\texttt{W[0]}_1$,$\texttt{W[3]}_2$) that lead to the observed Hamming weights are $\{(0,5),(0,10),(0,24),(0,192)\}$.

Intermediate value	HW	Combination of keys			
$\texttt{W[0]}_1$	0	0	0	0	0
$\texttt{W[3]}_2$	2	5	10	24	192
$\texttt{SubBytes(W[3]}_2\texttt{)}$	5	107	103	173	186
$\texttt{W[4]}_1$	5	107	103	173	186



Key schedule

Template-based DPA

- **Extending the template to use multiple traces in the attack phase.**
  - For each trace $\mathbf{t}'_i$, we have

$$p(k_j|\mathbf{t}'_i) = \frac{p(\mathbf{t}'_i|k_j) \cdot p(k_j)}{\sum_{l=1}^{K} \left(p(\mathbf{t}'_i|k_l) \cdot p(k_l)\right)}$$

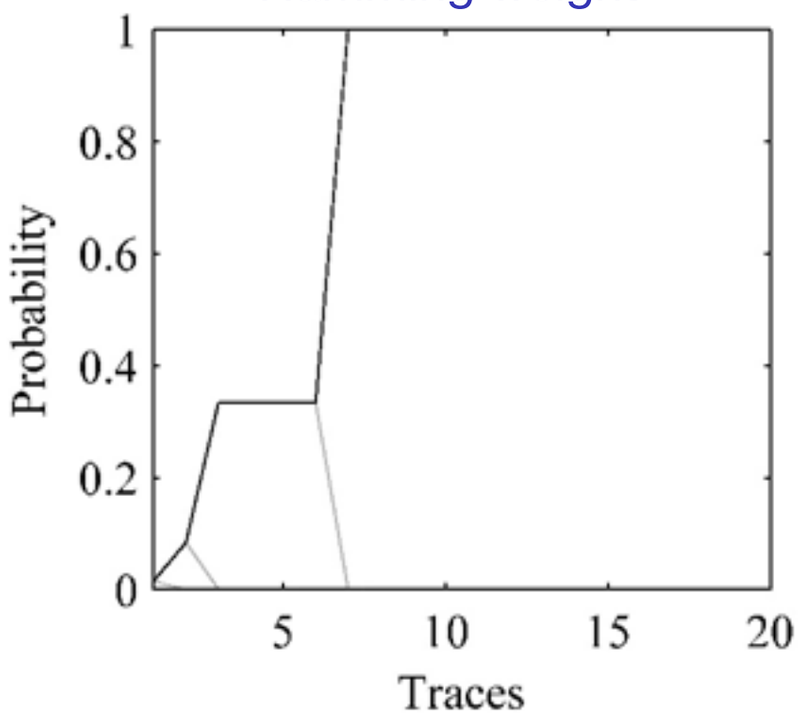
  - For multiple traces $\mathbf{T}$,

$$p(k_j|\mathbf{T}) = \frac{\left(\prod_{i=1}^{D} p(\mathbf{t}'_i|k_j)\right) \cdot p(k_j)}{\sum_{l=1}^{K} \left(\left(\prod_{i=1}^{D} p(\mathbf{t}'_i|k_l)\right) \cdot p(k_l)\right)}$$

Template-based DPA

- **Typically,** $p(k_j) = 1/K$
- $p(\mathbf{t}'_i|k_j) = p(\mathbf{t}'_i; \hbar_{v_{i,j}})$ **can be built using templates.**
- **In the attack phase, the probability of the correct key candidate grows as the number of traces increases.**

Template based on Hamming weight



Template based on LSB

